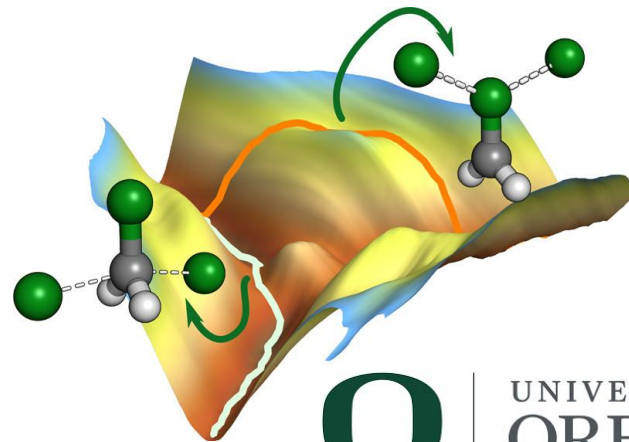
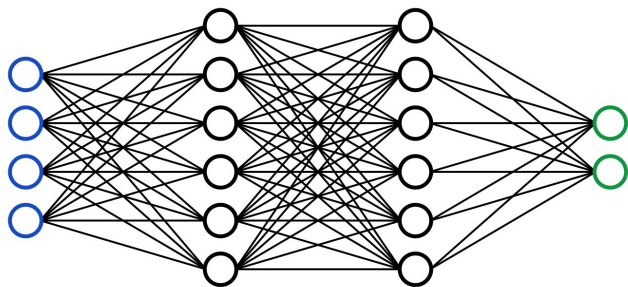


Machine Learning for Chemistry

Winter 2025

Topic 3: Regression & Gradient Descent



UNIVERSITY OF
OREGON

Instructor: Dr. Dhiman Ray

Assistant Professor, Department of Chemistry and Biochemistry

Overview of Topics

- Linear Regression
- Loss Function
- Gradient Descent
- Bias-variance trade-off and overfitting
- Regularization

References:

1. **Hands-on Machine Learning with Scikit-Learn, Keras, and TensorFlow** by Aurélien Géron (Chapter 4)
2. **Machine Learning with Pytorch and Scikit-Learn** by Raschka, Liu, and Mirjalili (Chapter 9 and part of Chapter 2)

Training Our First Machine Learning Model

In this lesson, we will build our first machine learning model step-by-step.

We will start with one of the simplest form of **Supervised Learning**

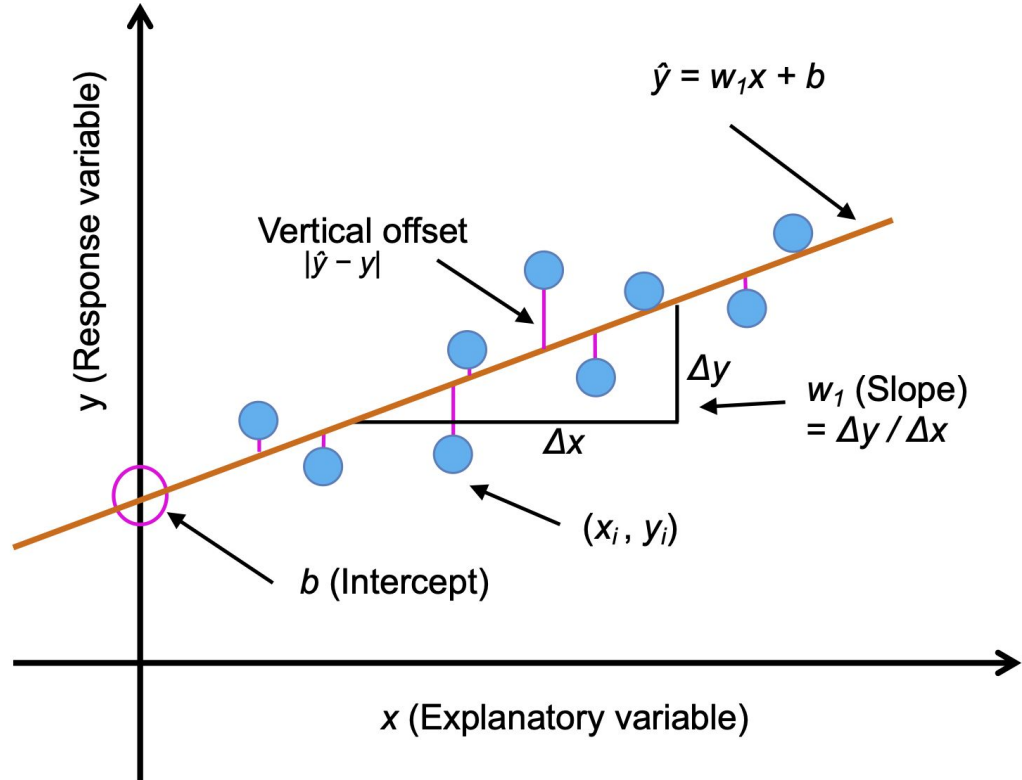
Supervised learning	> Labeled data > Direct feedback > Predict outcome/future
Unsupervised learning	> No labels/targets > No feedback > Find hidden structure in data
Reinforcement learning	> Decision process > Reward system > Learn series of actions

Simple Linear Regression

Simple (univariate) linear regression models the relationship between a single **feature** (explanatory variable, x) and a continuous-valued **target** (response variable, y).

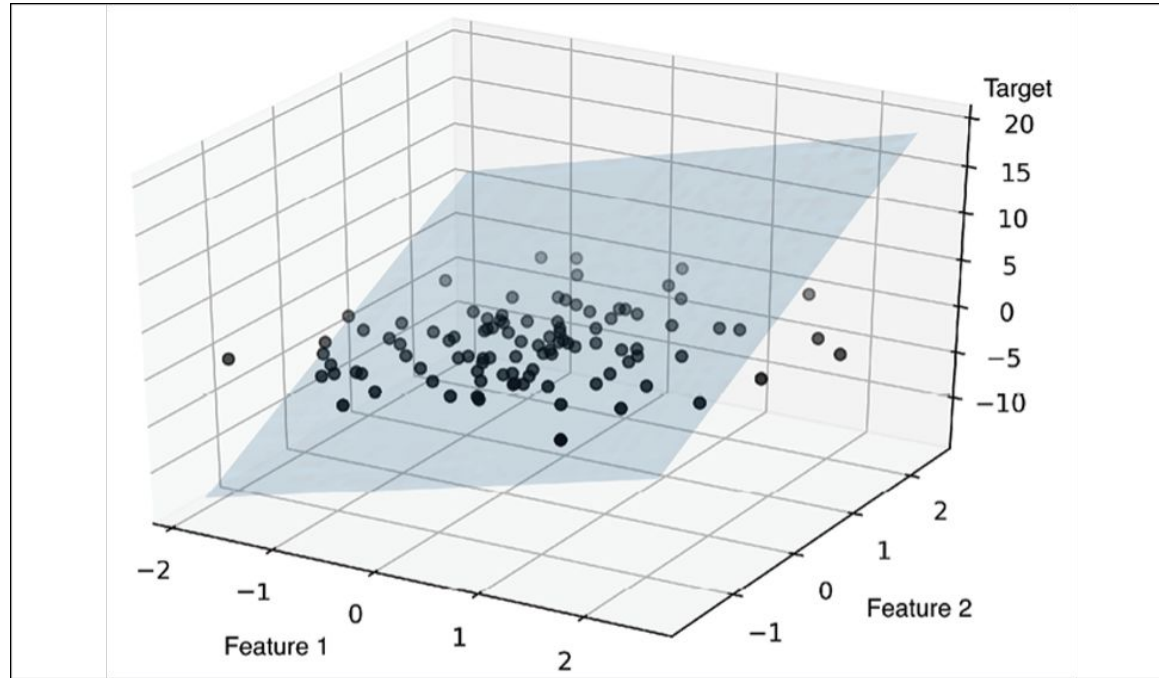
$$y = w_1x + b$$

Bias b , represents the y axis intercept and w_1 is the **weight** coefficient



Multiple Linear Regression

$$y = w_1x_1 + \dots + w_mx_m + b = \sum_{i=1}^m w_ix_i + b = w^T\mathbf{x} + b$$



Example: Ames Housing Dataset

Ames Housing dataset consists of 2,930 examples and 80 features. For simplicity, we will only work with a subset of the features:

	Overall Qual	Overall Cond	Total Bsmt SF	Central Air	Gr Liv Area	SalePrice
0	6	5	1080.0	Y	1656	215000
1	5	6	882.0	Y	896	105000
2	6	6	1329.0	Y	1329	172000
3	7	5	2110.0	Y	2110	244000
4	5	5	928.0	Y	1629	189900

Example: Ames Housing Dataset

Ames Housing dataset consists of 2,930 examples and 80 features. For simplicity, we will only work with a subset of the features:

The objective is to predict the Sale price (**target variable**) as a function of the other variables (**features**).

	Overall Qual	Overall Cond	Total Bsmt SF	Central Air	Gr Liv Area	SalePrice
0	6	5	1080.0	Y	1656	215000
1	5	6	882.0	Y	896	105000
2	6	6	1329.0	Y	1329	172000
3	7	5	2110.0	Y	2110	244000
4	5	5	928.0	Y	1629	189900

A Little Math Digression

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n = \boldsymbol{\theta}^T \mathbf{x},$$

- $\boldsymbol{\theta}$ is the model's *parameter vector*, containing the bias term θ_0 and the feature weights θ_1 to θ_n .
- $\mathbf{x}$ is the instance's *feature vector*, containing x_0 to x_n , with x_0 always equal to 1.
- $\boldsymbol{\theta} \cdot \mathbf{x}$ is the dot product of the vectors $\boldsymbol{\theta}$ and $\mathbf{x}$, which is of course equal to $\theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$.

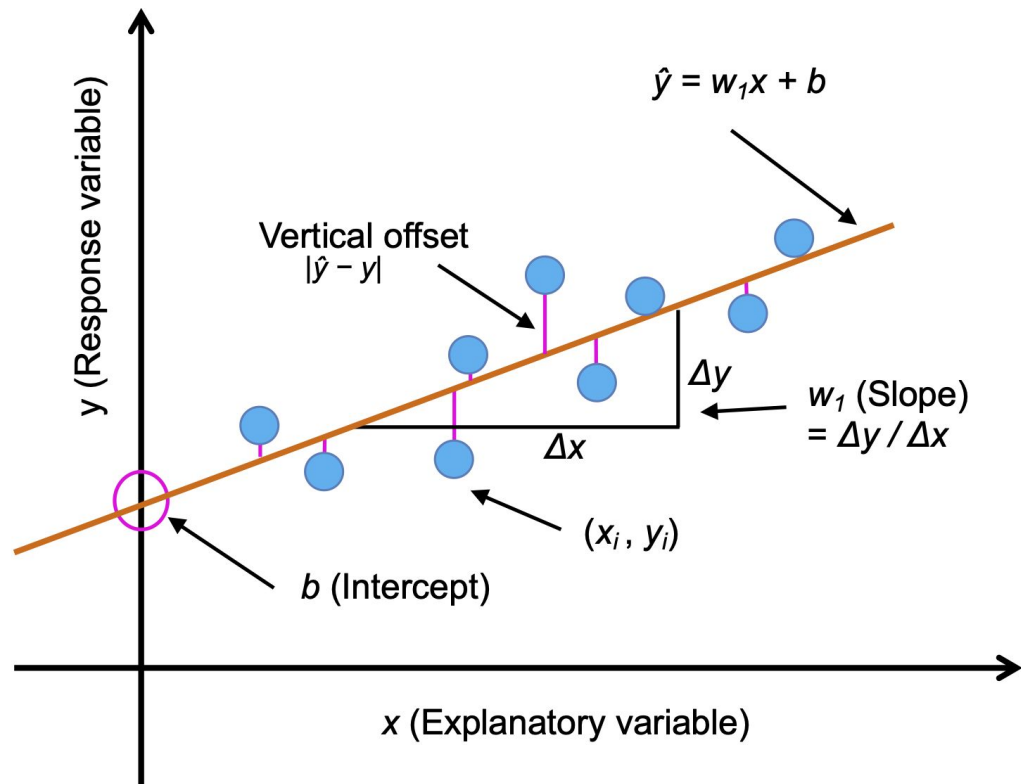
Loss Function

Our goal is find the parameter set that minimizes the difference between the predicted value and the true value of the response variable.

Mean Squared Error:

$$\begin{aligned}\text{MSE}(\boldsymbol{\theta}) &= \frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2 \\ &= \frac{1}{m} \sum_{i=1}^m (\boldsymbol{\theta}^T \mathbf{x}^{(i)} - y^{(i)})^2\end{aligned}$$

This is called a **Loss Function**



Important Properties of Loss Function

$$\begin{aligned}\text{MSE}(\boldsymbol{\theta}) &= \frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2 \\ &= \frac{1}{m} \sum_{i=1}^m (\boldsymbol{\theta}^T \mathbf{x}^{(i)} - y^{(i)})^2\end{aligned}$$

Why use square?: Avoid canceling of positive and negative deviations

Why not the absolute value i.e. |error|?: We minimize loss functions for training our model. Minimizations are performed by setting the derivatives to zero. So both our loss function and its first derivatives must be continuous.

The concept of **minimizing loss function is fundamental to machine learning** and this will appear again and again when we study e.g. classification, neural networks, etc.

Normal Equation

Goal: Find a combination of weights and biases that minimizes MSE Loss

But, for regression, there is an exact closed form expression for the parameters (Θ) that minimizes this loss function.

$$\hat{\theta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

This is known as normal equation, where vector $\mathbf{y}$ contains all target values.

$\mathbf{X}$ = matrix of all features and all data points; $\mathbf{x}$ = feature vector

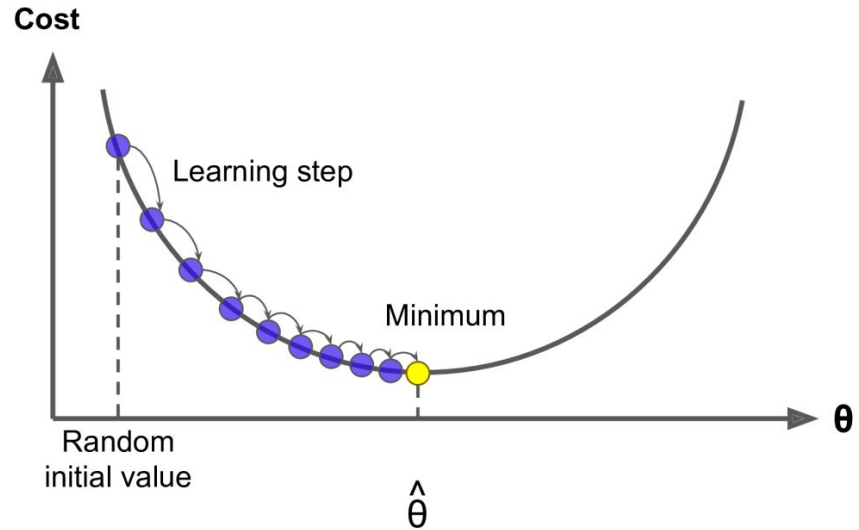
- But inverting matrices is computationally expensive: complexity $O(n^{2.4-3})$. Therefore cannot be used with large dataset.
- Closed form expression is not commonly available for other forms of machine learning algorithms.

We need to find a way that is both computationally scalable and more general.

Gradient Descent

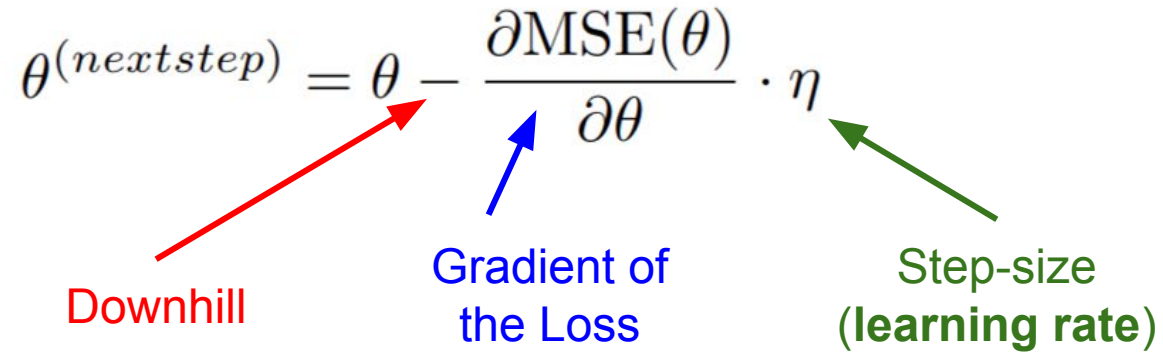
Suppose you are lost in the mountains in a dense fog; you can only feel the slope of the ground below your feet. A good strategy to get to the bottom of the valley quickly is to go downhill in the direction of the steepest slope.

Gradient Descent measures the local gradient of the error function with regards to the parameter vector θ , and it **goes in the direction of descending gradient**. Once the gradient is zero, you have reached a minimum!



Gradient Descent: Mathematics

In 1D the parameter θ is updated as:

$$\theta^{(nextstep)} = \theta - \frac{\partial \text{MSE}(\theta)}{\partial \theta} \cdot \eta$$


The diagram illustrates the gradient descent update equation with three color-coded annotations:

- Downhill** (red text): A red arrow points from this label to the minus sign in the equation, indicating the direction of the update.
- Gradient of the Loss** (blue text): A blue arrow points from this label to the fraction $\frac{\partial \text{MSE}(\theta)}{\partial \theta}$, representing the direction of the steepest descent of the loss function.
- Step-size (learning rate)** (green text): A green arrow points from this label to the learning rate η , which determines the size of the step taken in the direction of the negative gradient.

Gradient Descent: Mathematics

In higher dimensions: $\boldsymbol{\theta}^{(\text{next step})} = \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} \text{MSE}(\boldsymbol{\theta})$

$$\nabla_{\boldsymbol{\theta}} \text{MSE}(\boldsymbol{\theta}) = \begin{pmatrix} \frac{\partial}{\partial \theta_0} \text{MSE}(\boldsymbol{\theta}) \\ \frac{\partial}{\partial \theta_1} \text{MSE}(\boldsymbol{\theta}) \\ \vdots \\ \frac{\partial}{\partial \theta_n} \text{MSE}(\boldsymbol{\theta}) \end{pmatrix}$$

Numerical derivative calculations are error prone

But how to evaluate this efficiently and accurately?

Gradient Descent: Mathematics

$$\frac{\partial \text{MSE}(\boldsymbol{\theta})}{\partial \theta_k} = \frac{1}{m} \sum_{i=1}^m \frac{\partial}{\partial \theta_k} \left(\hat{y}^{(i)} - y^{(i)} \right)^2$$

Gradient Descent: Mathematics

$$\begin{aligned}\frac{\partial \text{MSE}(\boldsymbol{\theta})}{\partial \theta_k} &= \frac{1}{m} \sum_{i=1}^m \frac{\partial}{\partial \theta_k} \left(\hat{y}^{(i)} - y^{(i)} \right)^2 \\ &= \frac{1}{m} \sum_{i=1}^m \frac{\partial}{\partial \theta_k} \left(\sum_{j=0}^n \theta_j x_j^{(i)} - y^{(i)} \right)^2\end{aligned}$$

Gradient Descent: Mathematics

$$\begin{aligned}\frac{\partial \text{MSE}(\boldsymbol{\theta})}{\partial \theta_k} &= \frac{1}{m} \sum_{i=1}^m \frac{\partial}{\partial \theta_k} \left(\hat{y}^{(i)} - y^{(i)} \right)^2 \\ &= \frac{1}{m} \sum_{i=1}^m \frac{\partial}{\partial \theta_k} \left(\sum_{j=0}^n \theta_j x_j^{(i)} - y^{(i)} \right)^2 \\ &= \frac{2}{m} \sum_{i=1}^m \left(\sum_{j=0}^n \theta_j x_j^{(i)} - y^{(i)} \right) \cdot \frac{\partial}{\partial \theta_k} \left(\sum_{j=0}^n \theta_j x_j^{(i)} - y^{(i)} \right)\end{aligned}$$

Gradient Descent: Mathematics

$$\begin{aligned}\frac{\partial \text{MSE}(\boldsymbol{\theta})}{\partial \theta_k} &= \frac{1}{m} \sum_{i=1}^m \frac{\partial}{\partial \theta_k} \left(\hat{y}^{(i)} - y^{(i)} \right)^2 \\ &= \frac{1}{m} \sum_{i=1}^m \frac{\partial}{\partial \theta_k} \left(\sum_{j=0}^n \theta_j x_j^{(i)} - y^{(i)} \right)^2 \\ &= \frac{2}{m} \sum_{i=1}^m \left(\sum_{j=0}^n \theta_j x_j^{(i)} - y^{(i)} \right) \cdot \frac{\partial}{\partial \theta_k} \left(\sum_{j=0}^n \theta_j x_j^{(i)} - y^{(i)} \right)\end{aligned}$$

The derivative
survives only if $j = k$

Gradient Descent: Mathematics

$$\begin{aligned}\frac{\partial \text{MSE}(\boldsymbol{\theta})}{\partial \theta_k} &= \frac{1}{m} \sum_{i=1}^m \frac{\partial}{\partial \theta_k} \left(\hat{y}^{(i)} - y^{(i)} \right)^2 \\&= \frac{1}{m} \sum_{i=1}^m \frac{\partial}{\partial \theta_k} \left(\sum_{j=0}^n \theta_j x_j^{(i)} - y^{(i)} \right)^2 \\&= \frac{2}{m} \sum_{i=1}^m \left(\sum_{j=0}^n \theta_j x_j^{(i)} - y^{(i)} \right) \cdot \frac{\partial}{\partial \theta_k} \left(\sum_{j=0}^n \theta_j x_j^{(i)} - y^{(i)} \right) \\&= \frac{2}{m} \sum_{i=1}^m \left(\sum_{j=0}^n \theta_j x_j^{(i)} - y^{(i)} \right) \cdot x_k^{(i)}\end{aligned}$$

The derivative survives only if $j = k$

Gradient Descent: Mathematics

$$\begin{aligned}\frac{\partial \text{MSE}(\boldsymbol{\theta})}{\partial \theta_k} &= \frac{1}{m} \sum_{i=1}^m \frac{\partial}{\partial \theta_k} \left(\hat{y}^{(i)} - y^{(i)} \right)^2 \\&= \frac{1}{m} \sum_{i=1}^m \frac{\partial}{\partial \theta_k} \left(\sum_{j=0}^n \theta_j x_j^{(i)} - y^{(i)} \right)^2 \\&= \frac{2}{m} \sum_{i=1}^m \left(\sum_{j=0}^n \theta_j x_j^{(i)} - y^{(i)} \right) \cdot \frac{\partial}{\partial \theta_k} \left(\sum_{j=0}^n \theta_j x_j^{(i)} - y^{(i)} \right) \\&= \frac{2}{m} \sum_{i=1}^m \left(\sum_{j=0}^n \theta_j x_j^{(i)} - y^{(i)} \right) \cdot x_k^{(i)} \\&= \frac{2}{m} \sum_{i=1}^m \left(\hat{y}^{(i)} - y^{(i)} \right) \cdot x_k^{(i)}\end{aligned}$$

Gradient Descent: Mathematics

So θ_k can be updated as:

$$\theta_k^{(nextstep)} = \theta_k - \eta \cdot \sum_{i=1}^m \left(\hat{y}^{(i)} - y^{(i)} \right) \cdot x_k^{(i)}$$

A few things to note about this formula

- We do not need to calculate any (computationally expensive) derivative
- Highly parallelizable linear operation, each θ_k can be updated independently
- Easily generalizable to non-linear basis functions

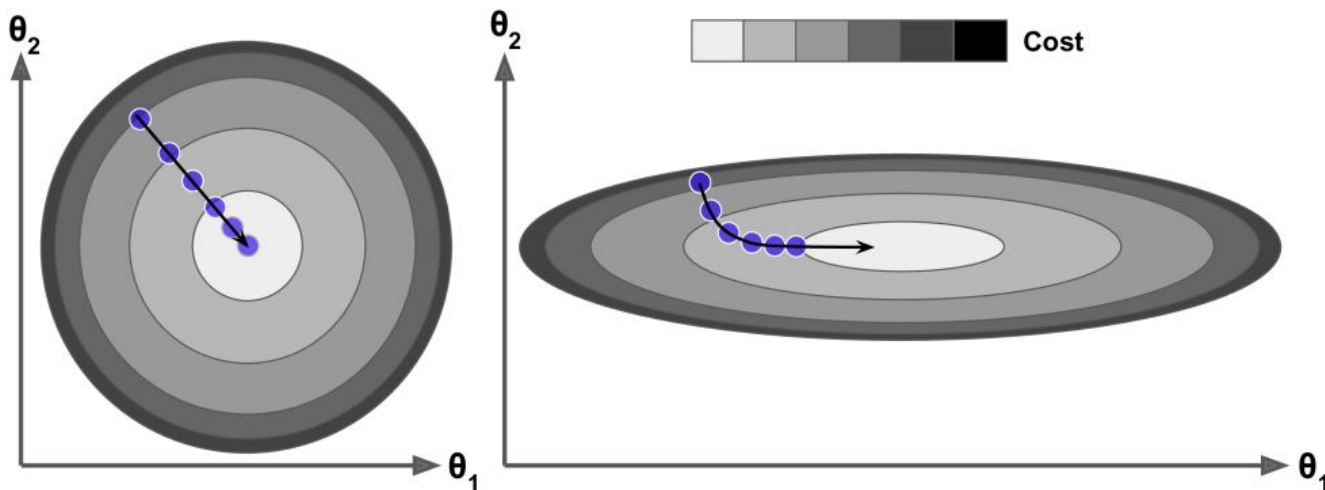
This equation is one of the key reason behind the AI revolution.

Feature Scaling

The magnitude of the features should be similar for optimal performance.

Shift the mean of each feature so that it is centered at zero and each feature

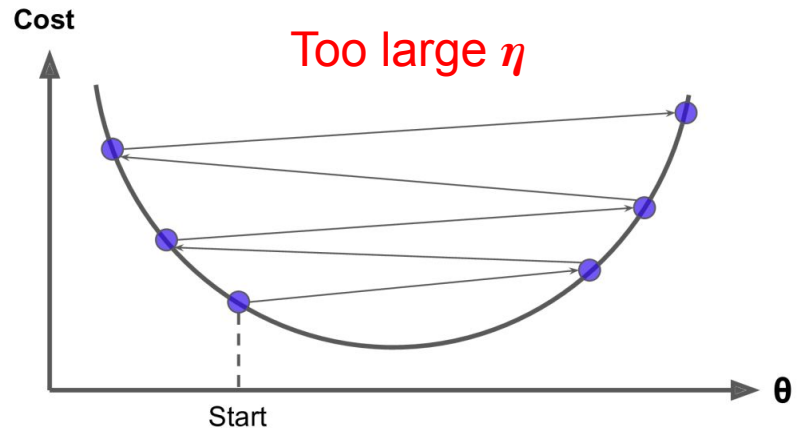
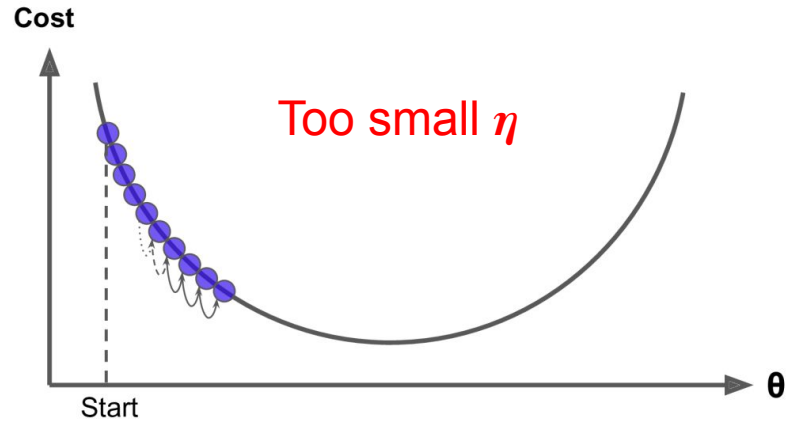
has a standard deviation of 1 (unit variance).
$$x'_j = \frac{x_j - \mu_j}{\sigma_j}$$



Learning rate (η)

If the learning rate is **too small**, then the algorithm will have to go through many iterations to converge, which will take a long time.

If the learning rate is **too high**, you might jump across the valley and end up on the other side, possibly even higher up than you were before. This might make **the algorithm diverge**, with larger and larger values, failing to find a good solution

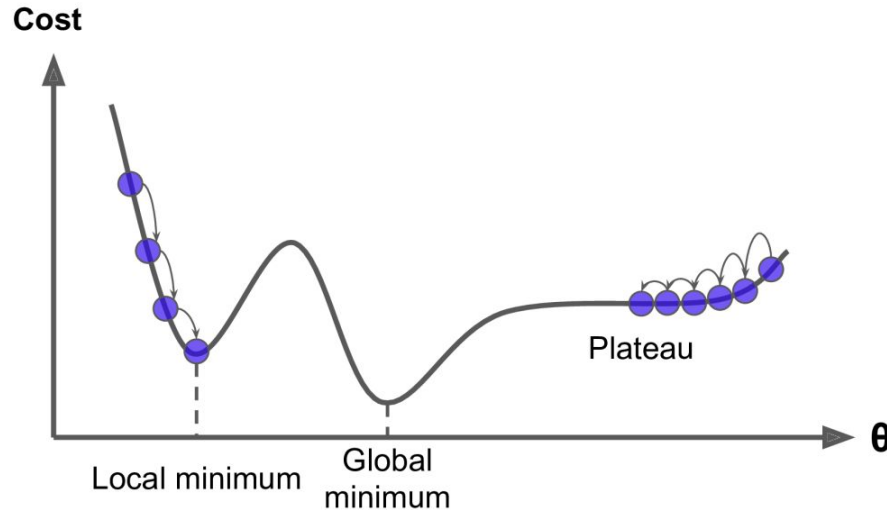


Limitations of Gradient Descent

1. Getting stuck in local minimum.
2. Takes very long time to cross the plateau

Not a big problem for linear regression: MSE Loss is a convex function

Can be a problem in other machine methods i.e. neural networks



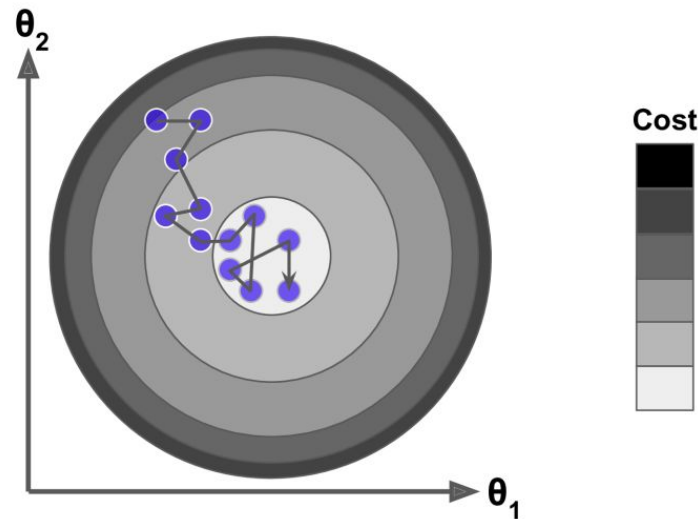
Stochastic Gradient Descent

Update weights for one sample at a time

$$\theta_k^{(nextstep)} = \theta_k - \eta \cdot \sum_{i=1}^m \left(\hat{y}^{(i)} - y^{(i)} \right) \cdot x_k^{(i)}$$

Quicker convergence: The error surface is noisier than in gradient descent. Can escape shallow local minima more readily

Can show large fluctuations of the loss function near minima.

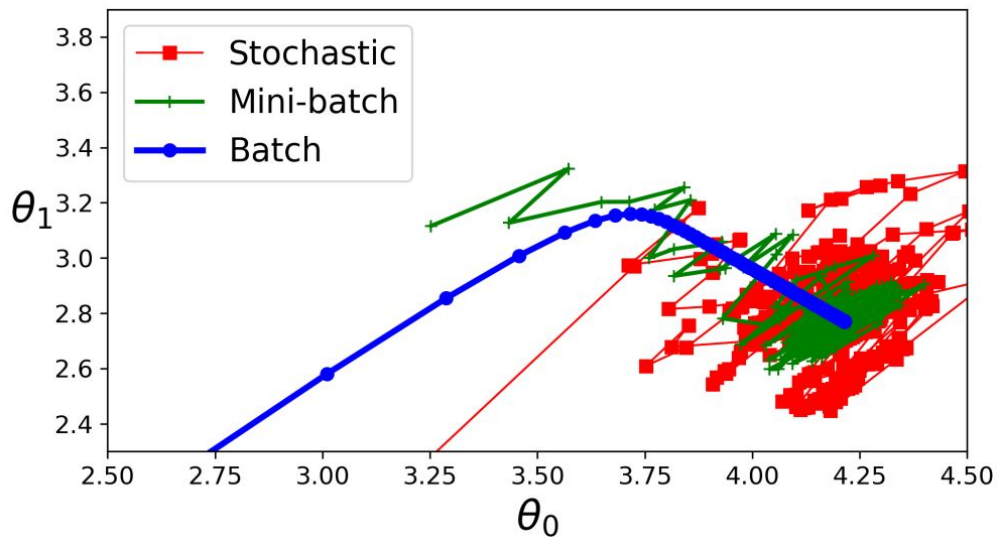


Mini-batch Gradient Descent

Intermediate between Normal (or batch) Gradient Descent and Stochastic Gradient Descent.

Randomly select a subset of data points instead of all or one.

Convergence is less erratic and we get the benefits of parallelization



Measuring the performance of linear regression

Mean Absolute Error (MAE):

$$MAE = \frac{1}{n} \sum_{i=1}^n |y^{(i)} - \hat{y}^{(i)}|$$

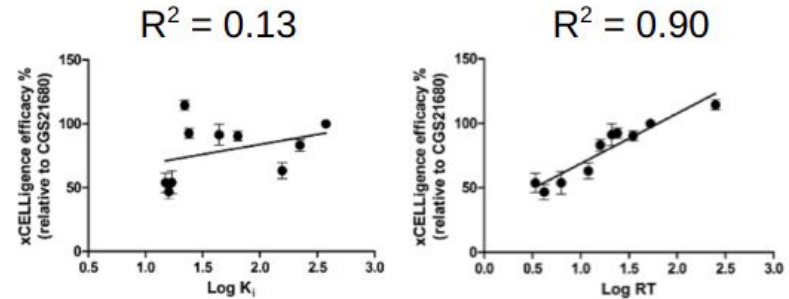
Coefficient of determination (R^2):

$$R^2 = 1 - \frac{SSE}{SST}$$

$$SSE = \sum_{i=1}^n (y^{(i)} - \hat{y}^{(i)})^2 \quad SST = \sum_{i=1}^n (y^{(i)} - \mu_y)^2$$

R^2 is a scaled version of MAE

$$R^2 = 1 - \frac{MSE}{Var(y)}$$

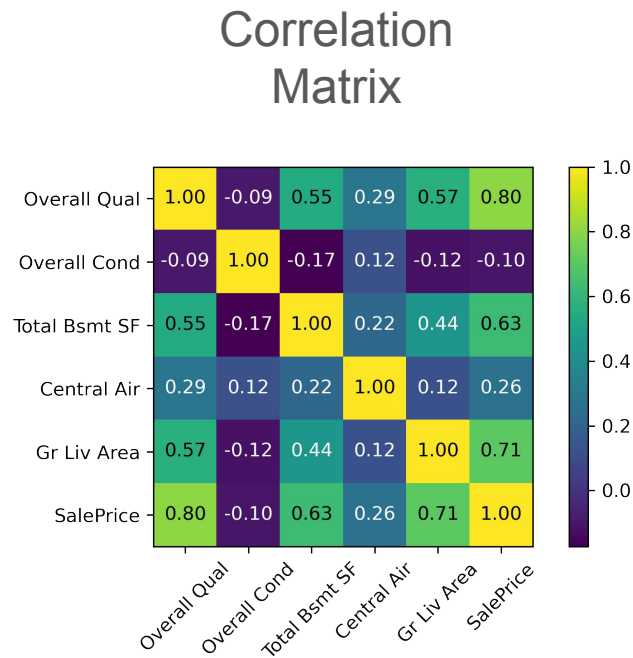
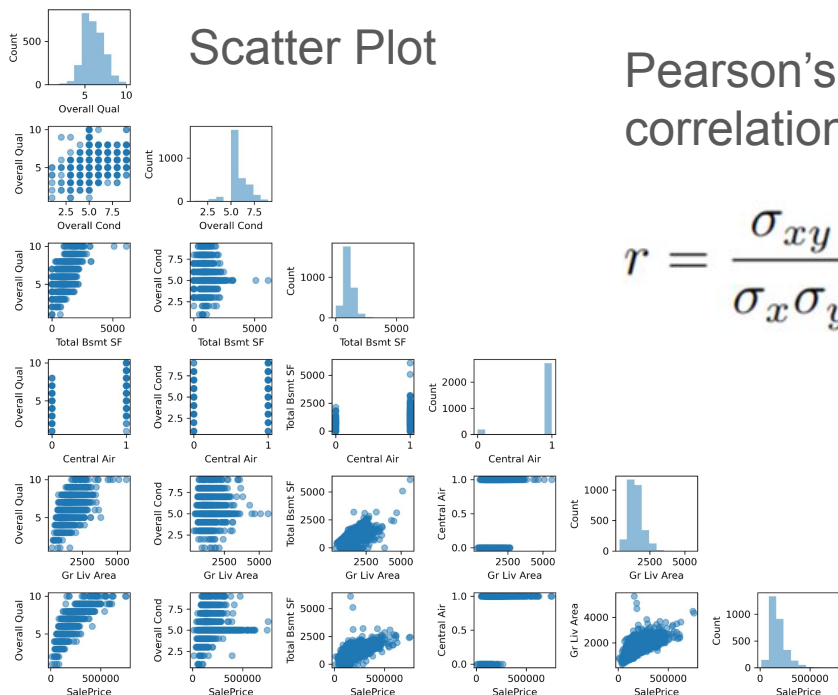


End of Part 1 of Regression

Go to Jupyter Notebook Exercise

Another hidden issue: Correlated Data

If some features are strongly correlated to each other, it is not useful to treat them as independent variables. Better to drop one of those.

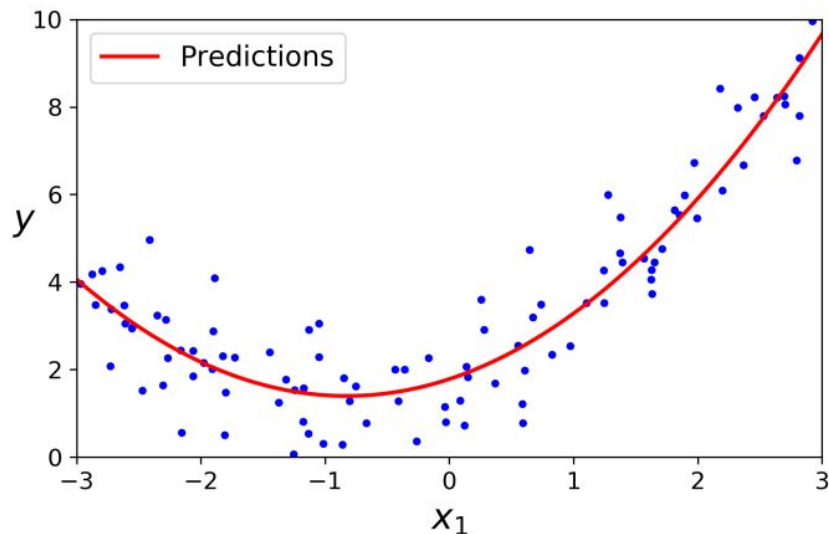


Polynomial Regression

For more complex relationships it is possible to do polynomial regression

$$y = w_1x + w_2x^2 + \dots + w_dx^d + b$$

Where d is the degree of the polynomial



True Function

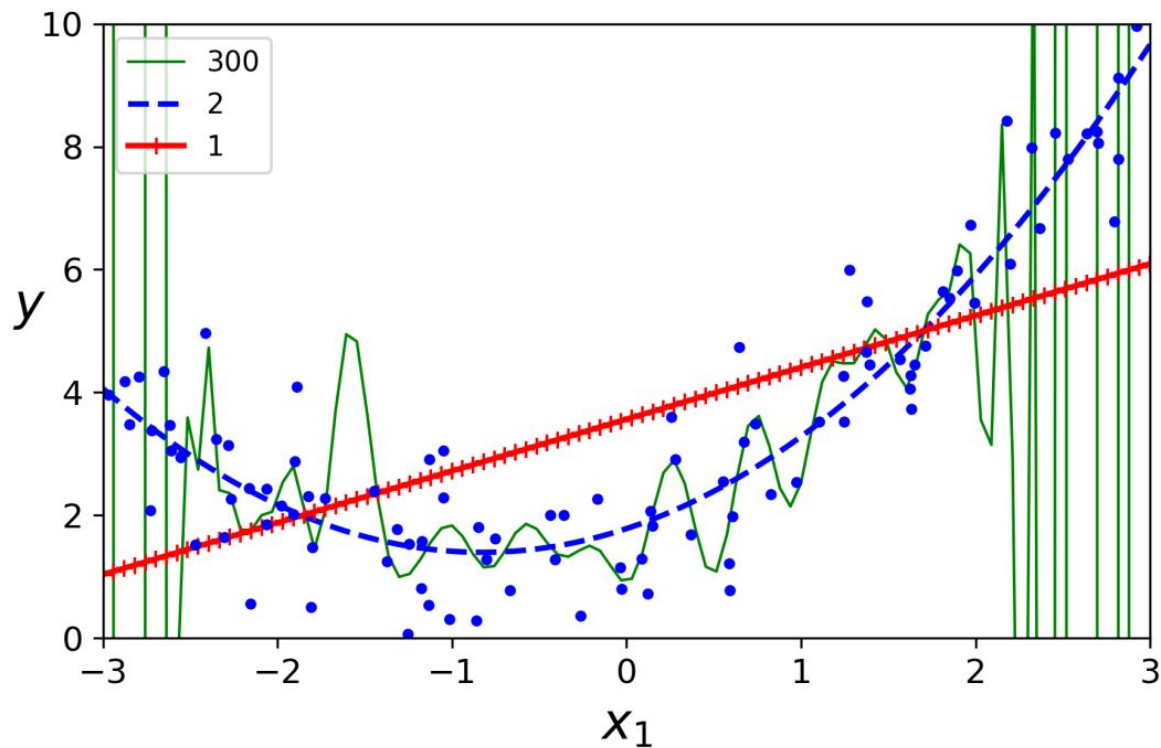
$$y = 0.5x_1^2 + 1.0x_1 + 2.0 + \text{Gaussian noise}$$

Estimate

$$\hat{y} = 0.56x_1^2 + 0.93x_1 + 1.78$$

Too High or Too Low Polynomial Degree

$$y = w_1x + w_2x^2 + \dots + w_dx^d + b$$



The Bias/Variance Tradeoff

An important theoretical result of statistics and Machine Learning is the fact that a model's generalization error can be expressed as the sum of three very different errors:

Bias

This part of the generalization error is due to wrong assumptions, such as assuming that the data is linear when it is actually quadratic. A high-bias model is most likely to underfit the training data.¹⁰

Variance

This part is due to the model's excessive sensitivity to small variations in the training data. A model with many degrees of freedom (such as a high-degree polynomial model) is likely to have high variance, and thus to overfit the training data.

Increasing a model's complexity will typically increase its variance and reduce its bias. Conversely, reducing a model's complexity increases its bias and reduces its variance. This is why it is called a tradeoff.

Bias/Variance Trade-off

High Bias, Low Variance:

Underfitting

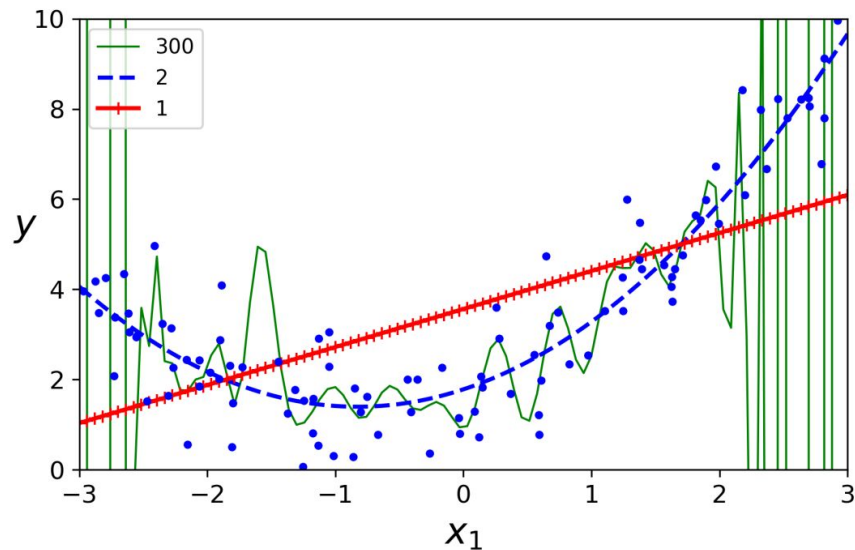
e.g. polynomial degree 1

Low Bias, High Variance:

Overfitting

e.g. polynomial degree 300

Optimal model: polynomial degree 2



Bias/Variance Trade-off

High Bias, Low Variance:

Underfitting

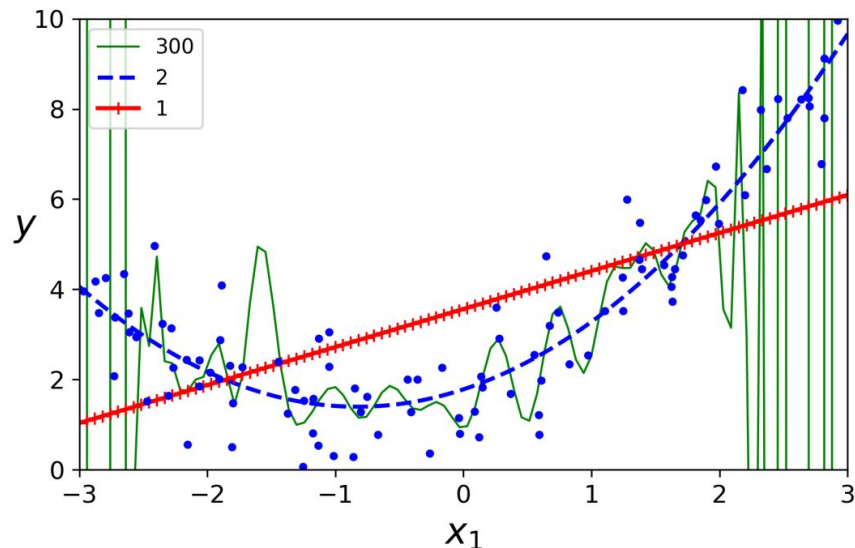
e.g. polynomial degree 1

Low Bias, High Variance:

Overfitting

e.g. polynomial degree 300

Optimal model: polynomial degree 2



But we do not know what is the optimal model to begin with!

Solution: **Regularization**

Regularization

A good way to reduce overfitting is to **constrain the model**: the fewer degrees of freedom it has, the harder it will be for it to overfit the data.

For example, **reduce the number of polynomial degrees**

We will discuss 4 common regularizations for regression:

- Ridge Regression (L2 penalty)
- Lasso Regression (L1 penalty)
- Elastic Net (both L1 and L2 penalty)
- Early Stopping (common in Neural Networks)

L2 penalty: Ridge Regression

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

New Loss Function: $\mathcal{L}(\boldsymbol{\theta}) = \text{MSE}(\boldsymbol{\theta}) + \alpha \cdot \frac{1}{2} \sum_{i=1}^n \theta_i^2$

L2 penalty: The L_2 norm of the weight vector (except the bias) is constrained

Prevents high weight values and reduces the gradient of predicted function.

This makes it smooth and avoids overfitting.

In regression, L2 regularization is called **ridge regression**

In neural networks L2 regularization is often called **weight decay**.

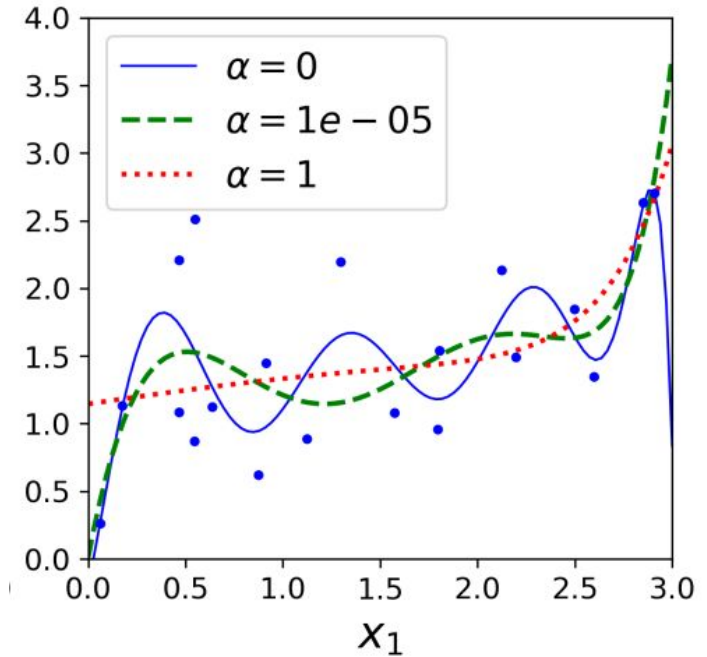
L2 penalty: Ridge Regression

Regularization parameter α :

Higher α leads to more bias and underfitting

Lower α leads to more variance and overfitting

$\alpha = 0$ leads to unregularized regression



L1 penalty: Lasso Regression

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

New Loss Function: $\mathcal{L}(\boldsymbol{\theta}) = \text{MSE}(\boldsymbol{\theta}) + \alpha \cdot \frac{1}{2} \sum_{i=1}^n |\theta_i|$

L1 penalty: The L_1 norm of the weight vector (except the bias) is constrained

Completely eliminates the weights of the least important features (i.e., set them to zero)

Automatically performs **feature selection** and outputs a **sparse model** (i.e., with few nonzero feature weights)

L1 penalty: Lasso Regression

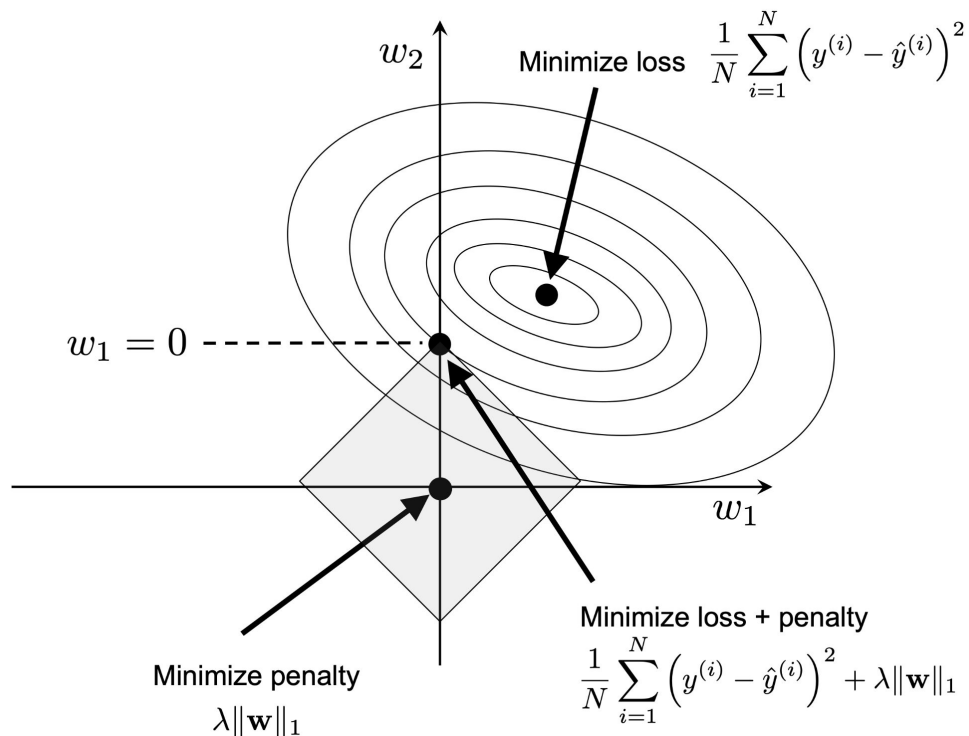
How does L1 penalty lead to a sparse model?

In this example: $\hat{y} = w_1x_1 + w_2x_2 + b$

The L1-penalty forces the algorithm to find a solution within the shaded region.

Optimal solution is where the rectangle intersects the MSE contour closest to its minimum

This is inevitably situated on either axes (corner of the rectangle)



Elastic Net Regularization

Intermediate (compromise) between Ridge and Lasso.

New Loss Function: $J(\boldsymbol{\theta}) = \text{MSE}(\boldsymbol{\theta}) + r\alpha \sum_{i=1}^n |\theta_i| + \frac{1-r}{2}\alpha \sum_{i=1}^n \theta_i^2$

Mix ratio (r): When $r = 0$, Elastic Net is equivalent to Ridge Regression, and when $r = 1$, it is equivalent to Lasso Regression.

Elastic Net Regularization

Intermediate (compromise) between Ridge and Lasso.

New Loss Function: $J(\boldsymbol{\theta}) = \text{MSE}(\boldsymbol{\theta}) + r\alpha \sum_{i=1}^n |\theta_i| + \frac{1-r}{2}\alpha \sum_{i=1}^n \theta_i^2$

Mix ratio (r): When $r = 0$, Elastic Net is equivalent to Ridge Regression, and when $r = 1$, it is equivalent to Lasso Regression.

Which regularization should I use?

You should always use some regularization. Generally L2 penalty is fine and commonly used. If you suspect that only a few features are actually useful, you should prefer Lasso or Elastic Net.

Elastic Net is preferred over Lasso since Lasso may behave erratically when the number of features is greater than the number of training instances

Early Stopping Regularization

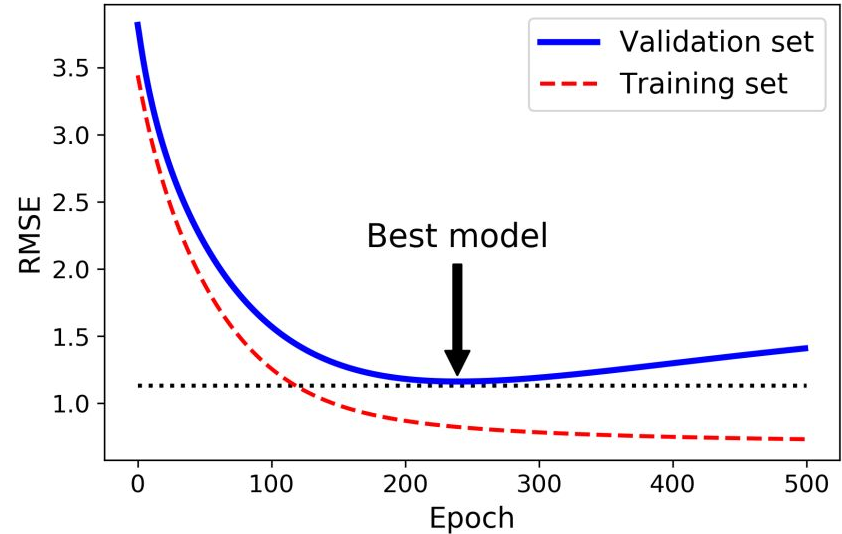
Learning Curves: Plot the error on the training and the validation set as a function of training rounds.

The training set error will keep decreasing

The validation set error will first decrease and then increase.

After a certain point the model starts to “memorize” the training data.

An overfitted model does not generalize well on the validation set.



Stop training when validation error reaches a minimum

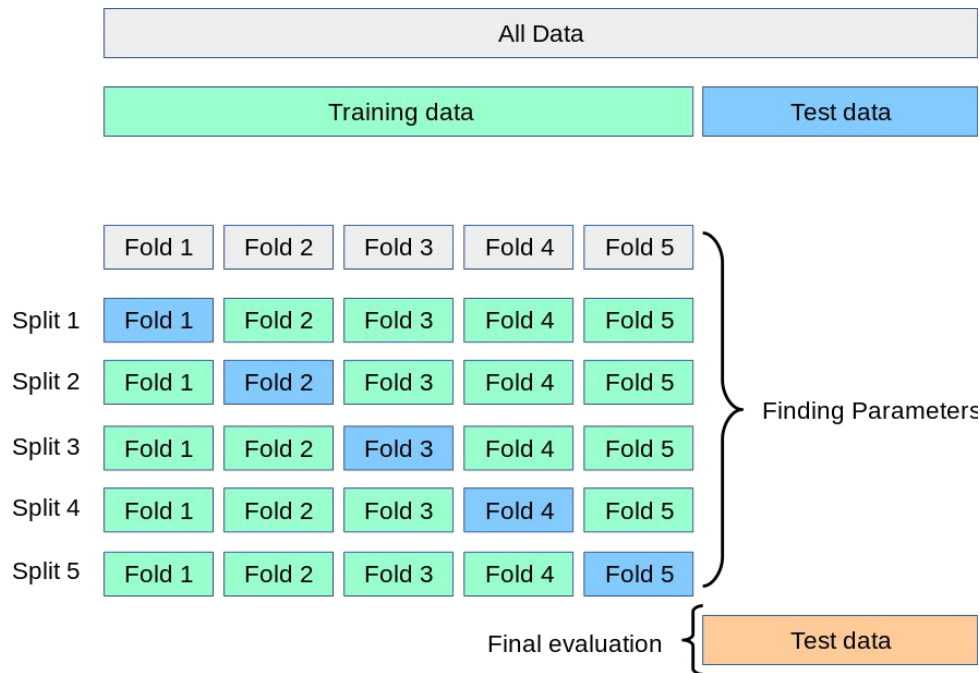
Hyperparameter tuning: cross validation

Training rate η , Regularization constant α , etc. are hyperparameters.

Hyperparameters can be optimized via k-fold cross validation.

Optimal hyperparameter values that yield a satisfying generalization, is estimated from evaluating the model performance on the test folds

Then we retrain the model on whole training dataset using those hyperparameters.

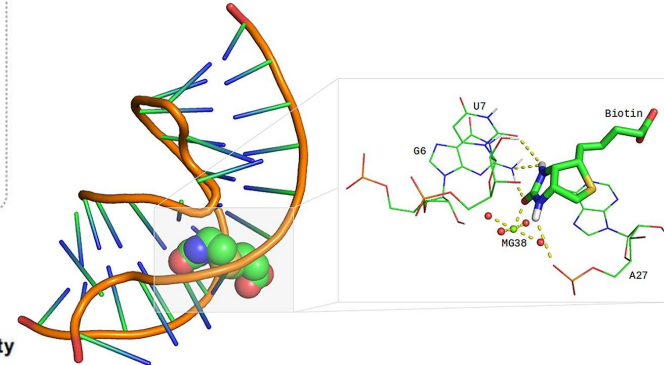
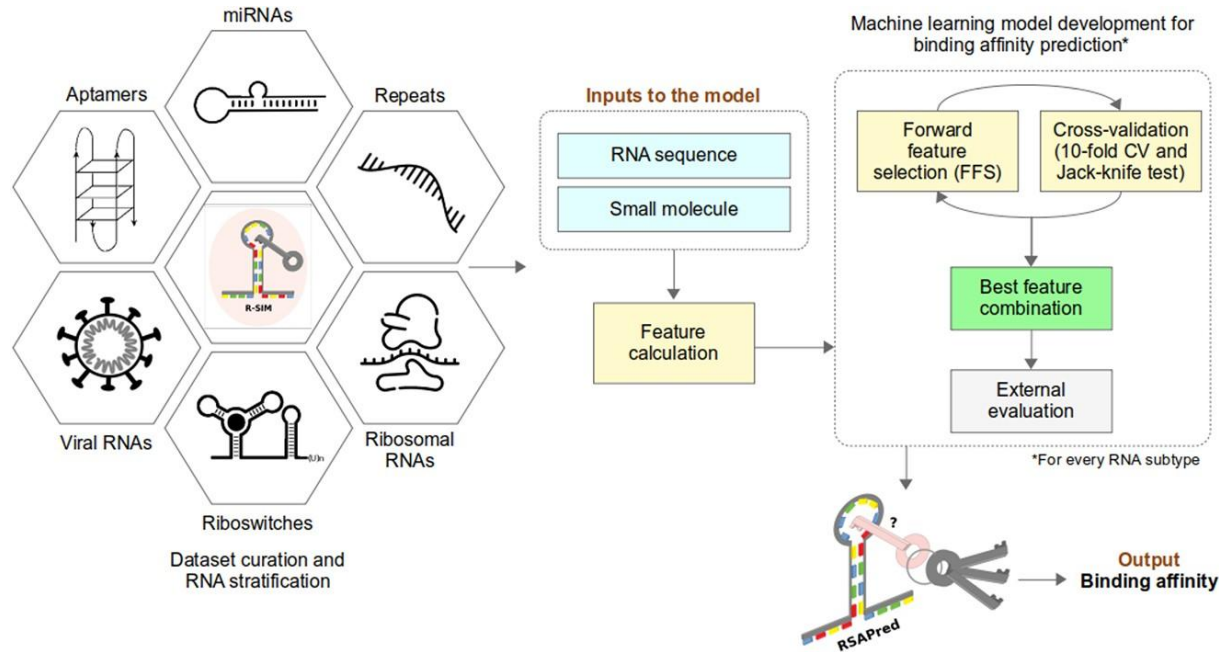


Example

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_k x_{ik} + \epsilon$$

Multivariate Linear Regression for RNA-drug binding affinity prediction.

Krishnan et al.
Brief. Bioinform. (2024)
<https://doi.org/10.1093/bib/bbae002>



Example

A data-driven interpretation of the stability of organic molecular crystals†

Rose K. Cersonsky, ^{lb}*^a Maria Pakhnova, ^{lb}^a Edgar A. Engel ^{lb}^b
and Michele Ceriotti ^{lb}^a

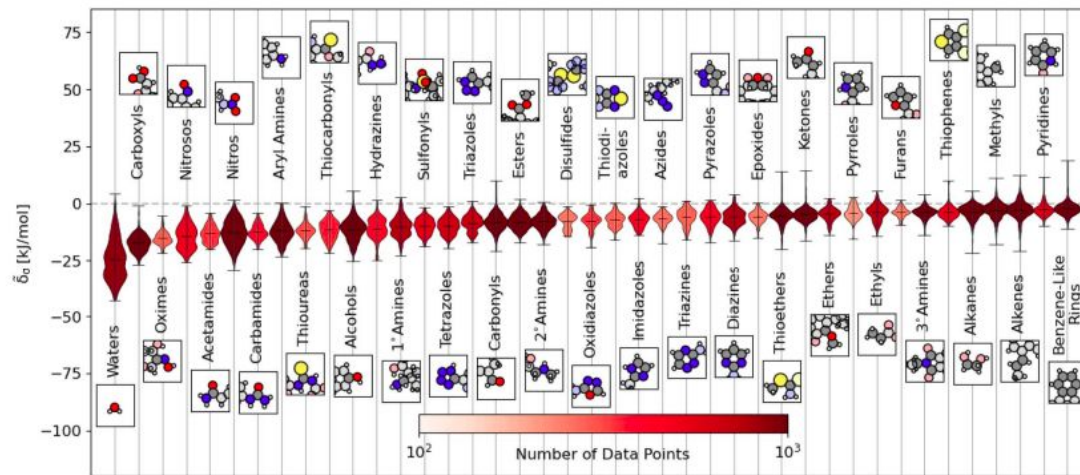


Fig. 2 Distribution of energetic contributions for different functional groups. For each functional group, we have taken the averaged remnant descriptor $x_v^{(s-9)}$ and computed the estimated contribution to the binding energy δ_e using the regressions detailed in eqn (10) and the filtering procedure in eqn (11). We have arranged the functional groups in order of average contribution, with a representative example shown above or below the violin plot with the functional group highlighted. We have limited this figure to those functional groups with more than 200 instances in the dataset (see Fig. S4† for all groups). The lines on each plot denote each group's extreme and mean contributions. The plots are colored by the number of examples within the dataset, ranging from 4 (pentazole) to 5313 (methyl groups). Wider sections of the violin plot represent a higher probability that members of the population will take on the given value; the skinnier sections represent a lower probability.

<https://pubs.rsc.org/en/content/articlehtml/2023/sc/d2sc06198h>

Example

Prediction of the Drug–Target Binding Kinetics for Flexible Proteins by Comparative Binding Energy Analysis

Ariane Nunes-Alves,* Fabian Ormersbach, and Rebecca C. Wade*



Cite This: *J. Chem. Inf. Model.* 2021, 61, 3708–3721



Read Online

Ensemble COMBINE

$$\log k_{\text{off}} \approx \sum_{i=1}^n w_i E_i + C$$

<https://doi.org/10.1021/acs.jcim.1c00639>

